A

Industrial Oriented Mini Project Report on

REAL TIME OBJECT TRACKING WITH VISION BASED AI ANALYTICS

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in the department of

COMPUTER SCIENCE AND ENGINEERING

by

S.SRUJANKUMAR	**23K81A05P4**
K.SAI VIGNESH	**23K81A05M7**
M.SIDDU	**23K81A05N3**
P.MINIHAS	**23K81A05Q8**

Under the Guidance of

Mrs.G.Bruhaspathi

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE Accredited by NBA & NAAC A+, ISO 21001:2018 Certified

Dhulapally, Secunderabad-500 100

www.smec.ac.in

APRIL - 2026

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE,

Accredited by NBA & NAAC A+, ISO 21001:2018 Certified

Dhulapally, Secunderabad - 500 100

Certificate

This is to certify that the Industrial Oriented Mini project entitled **"Real Time Object Tracking With Vision Based AI Analytics"** is being submitted by S.Srujan Kumar (23K81A05P4), K.Sai Vignesh (23K81A05M7), M,Siddu (23K81A05N3),P.Minihas (23K81A05Q8) in the partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** is a bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of the Guide
Mrs. G.BRUHASPATHI
Assistant Professor
Department of CSE

Signature of the HOD
Dr. K. SURESH
Professor and Head of the Department
Department of CSE

Internal Examiner **External Examiner**

Place:

Date:

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE,

Accredited by NBA & NAAC A+, ISO 21001:2018 Certified

Dhulapally, Secunderabad - 500 100

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the students of "**Bachelor of Technology** from the **Department of COMPUTER SCIENCE AND ENGINEERING"**, session: 2023 - 2027**, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad,** hereby declare that the work presented in this IOM project work entitled **Real Time object Tracking with Vision Based AI Analytics** is the outcome of our own bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report have not been submitted in any university for the award of any degree.

S. Srujan kumar	23K81A05P4
K. Sai Vignesh	23K81A05M7
M. Siddu	23K81A05N3
P. Minihas	23K81A05Q8

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. KASA RAVINDRA**, Professor and Director, St. Martin's Engineering College, Dhulapally, Secunderabad, for permitting us to undertake this project.

We are also thankful to **Dr. K. SURESH** , Head of the Department, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator **Dr. G. BRUSHASPATHI**, Assistant Professor, Computer Science and Engineering department for her valuable support.

We would like to express our sincere gratitude and indebtedness to our project supervisor **P.SWETHA**, Assistant Professor, Computer Science and Engineering, St. Martins Engineering College, Dhulapally, for her support and guidance throughout our project.

Finally, we express thanks to all those who have helped us directly or indirectly towards successful completion this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement.

S. Srujan Kumar	23K81A05P4
K. Sai Vignesh	23K81A05M7
M. Siddu	23K81A05N3
P. Minihas	23K81A05Q8

ABSTRACT

In many real-world environments, users rely on visual monitoring systems to observe and analyze objects from images, videos, or live camera feeds; however, traditional systems require continuous manual observation, making the process time-consuming, inefficient, and prone to human error. The operator must identify objects, analyze activities, and interpret results from multiple sources, which reduces accuracy and consistency in monitoring tasks. Therefore, the purpose of this project is to utilize artificial intelligence–based computer vision and deep learning techniques to automatically detect and track objects, and to develop a real-time object detection system with analytics capabilities for efficient monitoring and decision-making. The system is implemented using the YOLOv8 model along with OpenCV to process webcam, image, and video inputs, enabling accurate object detection, classification, and real-time analytics such as object count, confidence level, and processing speed (FPS). A web-based dashboard is integrated to provide users with an interactive interface for visualizing detection results and accessing analytics instantly. The implementation of this system reduces the complexity of manual monitoring and improves efficiency and accuracy in object detection tasks. Furthermore, the project demonstrates the practical application of artificial intelligence in computer vision and provides a scalable foundation for future applications such as surveillance systems, smart city solutions, and industrial automation.

LIST OF FIGURES

LIST OF ACRONYMS AND DEFINITIONS

S.NO	ACRONYM	DEFINITION
01.	CNN	Convolutional Nerual Networks
02.	AI	Artificial Intelligence
03.	FLASK	Python Web Framework
04.	YOLO	You Only Look Once
05.	FPS	Frames per Second
06.	OPENCV	Open Source Computer Vision Library
07.	HTML	HyperText Markup Language
08.	DL	Deep Learning
09.	JSON	JavaScript Object Notation
10.	API	Application Program Interface

CONTENTS

CHAPTER 1
INTRODUCTION

The proposed system is developed using a modern web-based architecture that ensures scalability, performance, and reliability. It follows a client-server model where the frontend, backend, and AI services interact seamlessly to deliver a smooth and efficient user experience. The system is capable of handling multiple inputs such as webcam, image, and video simultaneously while maintaining real-time processing and responsiveness..

The frontend of the system is built using HTML, CSS, and JavaScript, which provides a dynamic and responsive user interface. It enables smooth navigation across different modules such as webcam detection, image upload, video processing, and analytics dashboard. The backend is developed using Python and Flask, which handle API requests, manage application logic, and ensure proper communication between system components.

The system uses OpenCV for image and video processing and integrates the YOLOv8 deep learning model for object detection. The model performs feature extraction, bounding box prediction, and classification of objects with high accuracy. Detection results such as object labels, confidence scores, and bounding boxes are generated in real time and displayed on the user interface..

Temporary data such as object count, detection results, and performance metrics like frames per second (FPS) are managed within the system to provide real-time analytics. The system ensures efficient processing and smooth data flow without the need for complex database management.

Overall, the system configuration provides a robust, efficient, and scalable environment that supports real-time object detection, intelligent analytics, and seamless user interaction, making it a powerful solution for modern computer vision applications.

1.1 OBJECTIVE

The primary objective of the proposed system is to develop a real-time object detection and tracking application using advanced artificial intelligence and computer vision techniques. The system aims to accurately identify and classify objects from different input sources such as webcam, images, and video files, ensuring efficient and automated monitoring.

Another important objective is to reduce human effort and dependency on manual observation by automating the detection process. The system continuously analyzes visual data and provides instant results, minimizing errors and improving overall efficiency. This helps in applications where continuous monitoring is required.

The system also aims to achieve high accuracy and speed by using the YOLOv8 deep learning model. By processing the entire image in a single step, the system ensures faster detection while maintaining reliable results. This makes it suitable for real-time applications where quick decision-making is essential.

Additionally, the project focuses on providing a user-friendly web interface that allows users to easily interact with the system. The interface supports multiple functionalities such as uploading images, processing videos, and viewing real-time webcam detection along with analytics like object count and performance metrics.

Finally, the objective of the system is to create a scalable and flexible solution that can be extended for future applications. The system can be enhanced with additional features such as cloud integration, multi-camera support, and advanced analytics, making it useful for various real-world scenarios like surveillance, traffic monitoring, and smart systems.

1.2 OVERVIEW

The Real-Time Object Detection System is a full-stack web application that allows users to detect and analyze objects using artificial intelligence. Users can provide input through webcam, image, or video, and the system processes the input to identify and classify objects in real time with bounding boxes and labels.

The platform also enables users to view detection results along with analytics such as object count, confidence scores, and frames per second (FPS). The system provides an interactive dashboard where users can easily switch between different input modes and monitor performance metrics effectively.

The system architecture is designed to be efficient and scalable, using a client-server model for communication between frontend and backend components. The backend handles processing and detection tasks, while the frontend displays results in a user-friendly interface. The system ensures smooth data flow and real-time response for better user experience.

The Real-Time Object Detection System is designed to provide an intelligent and automated monitoring solution by integrating artificial intelligence with computer vision techniques. The platform allows users to analyze visual data instantly, eliminating the need for manual observation and improving efficiency in detection tasks.

By leveraging advanced deep learning techniques such as the YOLOv8 algorithm, the system is capable of delivering fast and accurate object detection results. This enables users to monitor environments effectively and obtain meaningful insights, making the system suitable for applications such as surveillance, traffic monitoring, and smart analytics systems.

CHAPTER 2
LITERATURE SURVEY

The rapid advancement of artificial intelligence and computer vision technologies has led to significant developments in object detection and tracking systems. Researchers have explored various approaches using machine learning and deep learning techniques to analyze visual data from images and videos. These systems aim to automate monitoring processes, reduce human effort, and improve accuracy in applications such as surveillance, traffic monitoring, and industrial automation

One of the most widely used approaches in object detection is the YOLO (You Only Look Once) algorithm. Earlier methods such as R-CNN and Fast R-CNN provided good accuracy but were computationally expensive and slow for real-time applications. YOLO introduced a single-stage detection approach, enabling faster processing by analyzing the entire image in one pass. Recent versions like YOLOv8 have further improved detection accuracy, speed, and performance, making them suitable for real-time object detection systems.

By integrating OpenCV with deep learning models, researchers have developed systems capable of detecting and tracking objects continuously in dynamic environments. These systems can handle multiple objects and provide visual outputs with bounding boxes and labels.Despite the progress in object detection technologies, several challenges still exist, such as variations in lighting conditions, object occlusion, and real-time processing limitations. Researchers continue to improve model performance, reduce latency, and enhance detection accuracy in complex environments..

The proposed Real-Time Object Detection and Tracking System builds upon these advancements by integrating YOLOv8 for detection, OpenCV for frame processing, and a Flask-based web application for user interaction. By combining these technologies, the system provides an efficient, accurate, and scalable solution for real-time object monitoring and analytics

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 EXISTING SYSTEM

Most existing object detection and monitoring systems rely on manual observation or basic surveillance tools that provide limited functionality such as video recording and simple playback. While these systems offer basic monitoring support, they fail to provide automated analysis and real-time detection capabilities. Users are often required to continuously watch video feeds or review recorded footage, which leads to inefficiency and increased chances of missing important events.

Disadvantages

- Monitoring depends heavily on human observation, making it time-consuming and less efficient..
- High possibility of missing important events due to human error or lack of attention.
- No automatic object detection or classification capabilities..
- Lack of real-time analytics such as object count and performance metrics
- Requires manual review of recorded videos, which is time-consuming.
- Not scalable for large-scale monitoring systems with multiple inputs.
- Limited use of artificial intelligence for intelligent decision-making and automation.

3.1 PROPOSED SYSTEM

The proposed system is a Real-Time Object Detection System that integrates artificial intelligence and computer vision techniques into a single unified platform. It is designed to provide an automated, accurate, and efficient solution for detecting and tracking objects from different input sources such as webcam, images, and videos. The system uses advanced deep learning algorithms to perform real-time analysis and deliver meaningful insights through an interactive web-based interfac.

Advantages

- Provides real-time object detection using advanced AI and YOLOv8 algorithm.
- Supports multiple input types such as webcam, image, and video processing.
- Automatically detects and classifies multiple objects simultaneously.
- Reduces human effort by automating monitoring and detection tasks.
- Provides real-time analytics such as object count and frames per second (FPS).
- Ensures high accuracy and fast processing using deep learning techniques.
- Offers a user-friendly interface for easy interaction and visualization.
- Improves efficiency in monitoring systems through intelligent automation..
- Can be extended for various applications such as surveillance, traffic monitoring, and smart systems..

.

3.2 SYSTEM CONFIGURATION

The proposed system is developed using a modern web-based architecture that ensures scalability, performance, and reliability. It follows a client-server model where the frontend, backend, and AI services interact seamlessly to deliver a smooth and efficient user experience. The system is capable of handling multiple inputs such as webcam, image, and video simultaneously while maintaining real-time processing and responsiveness.

The frontend of the system is built using HTML, CSS, and JavaScript, which provides a dynamic and responsive user interface. It enables smooth navigation across different modules such as webcam detection, image upload, video processing, and analytics dashboard. The backend is developed using Python and Flask, which handle API requests, manage application logic, and ensure proper communication between system components.

The system uses OpenCV for image and video processing and integrates the YOLOv8 deep learning model for object detection. The model performs feature extraction, bounding box prediction, and classification of objects with high accuracy. Detection results such as object labels, confidence scores, and bounding boxes are generated in real time and displayed on the user interface.

Temporary data such as object count, detection results, and performance metrics like frames per second (FPS) are managed within the system to provide real-time analytics. The system ensures efficient processing and smooth data flow without the need for complex database management.

Overall, the system configuration provides a robust, efficient, and scalable environment that supports real-time object detection, intelligent analytics, and seamless user interaction, making it a powerful solution for modern computer vision applications.

CHAPTER 4
SYSTEM REQUIREMENTS & SPECIFICATIONS

4.1 YOLOv8 Algorithm

The YOLOv8 (You Only Look Once version 8) is a state-of-the-art object detection algorithm based on deep learning. It is widely used for real-time applications due to its speed and accuracy. Unlike traditional methods that process images in multiple stages, YOLO processes the entire image in a single pass, making it highly efficient..

The database is organized into several collections, including Users, Courses, GeneratedContent, Progress, Flashcards, Quizzes, and Documents. The Users collection stores user information such as name, email, authentication details, and preferences. The Courses collection maintains AI-generated course structures, including topics, modules, and content. The GeneratedContent collection stores dynamically created materials such as summaries and explanations generated by the system.

4.2 System Design

The system of the proposed Real-Time Object Detection System is designed to provide an efficient and scalable solution for processing visual data using artificial intelligence and computer vision techniques. The architecture consists of multiple interconnected components that work together to perform object detection, tracking, and analytics in real time. It is structured into different layers to ensure modularity, flexibility, and smooth communication between system components.

The architecture can be broadly divided into three main layers: User Layer, Application Layer, and AI Processing Layer. Each layer performs specific functions and contributes to the overall working of the system.

4.3 DESIGN

4.3.1 System Architecture

The system architecture of the proposed Real-Time Object Detection System is designed to provide an efficient and scalable solution for processing visual data using artificial intelligence and computer vision techniques. The architecture consists of multiple interconnected components that work together to perform object detection, tracking, and analytics in real time. It is structured into different layers to ensure modularity, flexibility, and smooth communication between system components. The architecture can be broadly divided into three main layers: User Layer, Application Layer, and AI Processing Layer Each layer performs specific functions and contributes to the overall working of the system.

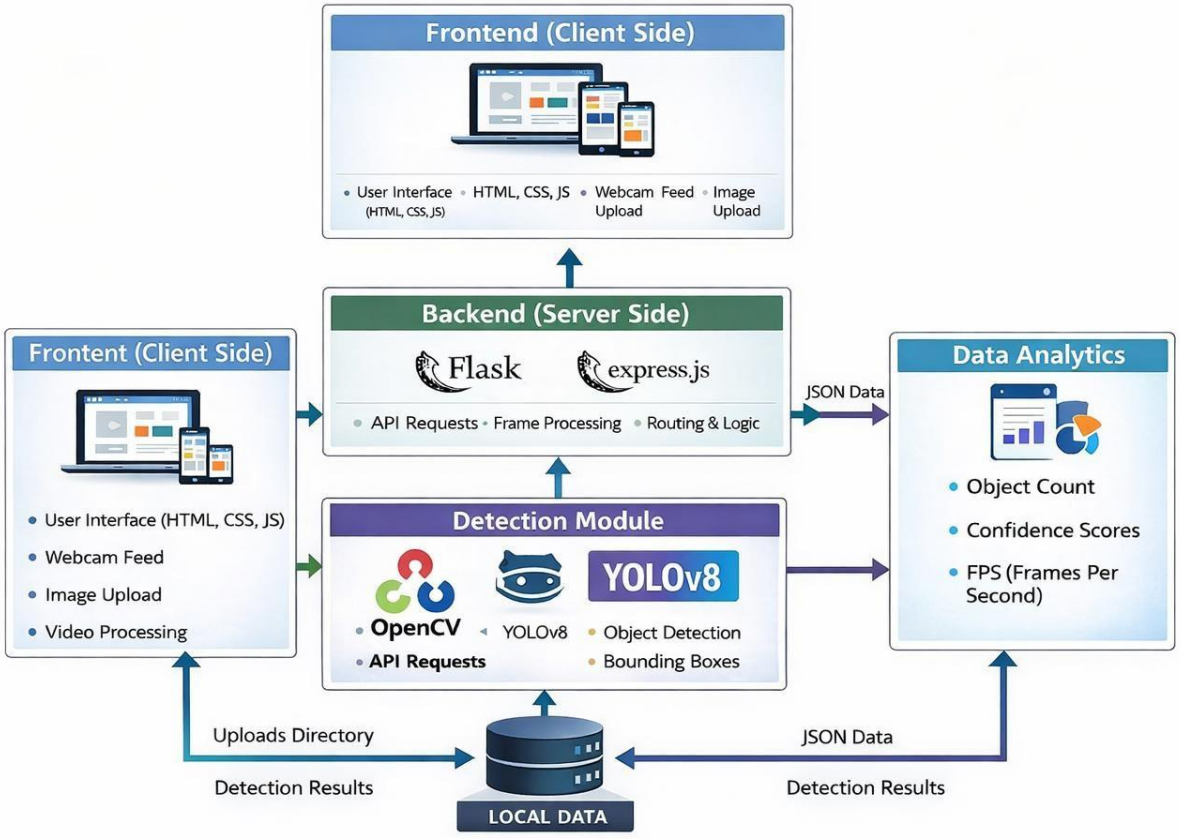

Fig:4.1 System Architecture

4.3.2 Data Flow Diagram

The Data Flow Diagram (DFD) represents how the system flow description represents how data moves through the Real-Time Object Detection System and how different components interact during the detection process. It provides a clear understanding of how input data from sources such as webcam, image, or video is processed, analyzed, and converted into meaningful detection results. The proposed system follows a structured flow where input frames are captured, processed using computer vision techniques, and analyzed by the YOLOv8 model to identify and classify objects. The detected results are then displayed to the user along with additional analytics such as object count and frame rate, ensuring accurate and real-time performance throughout the system..

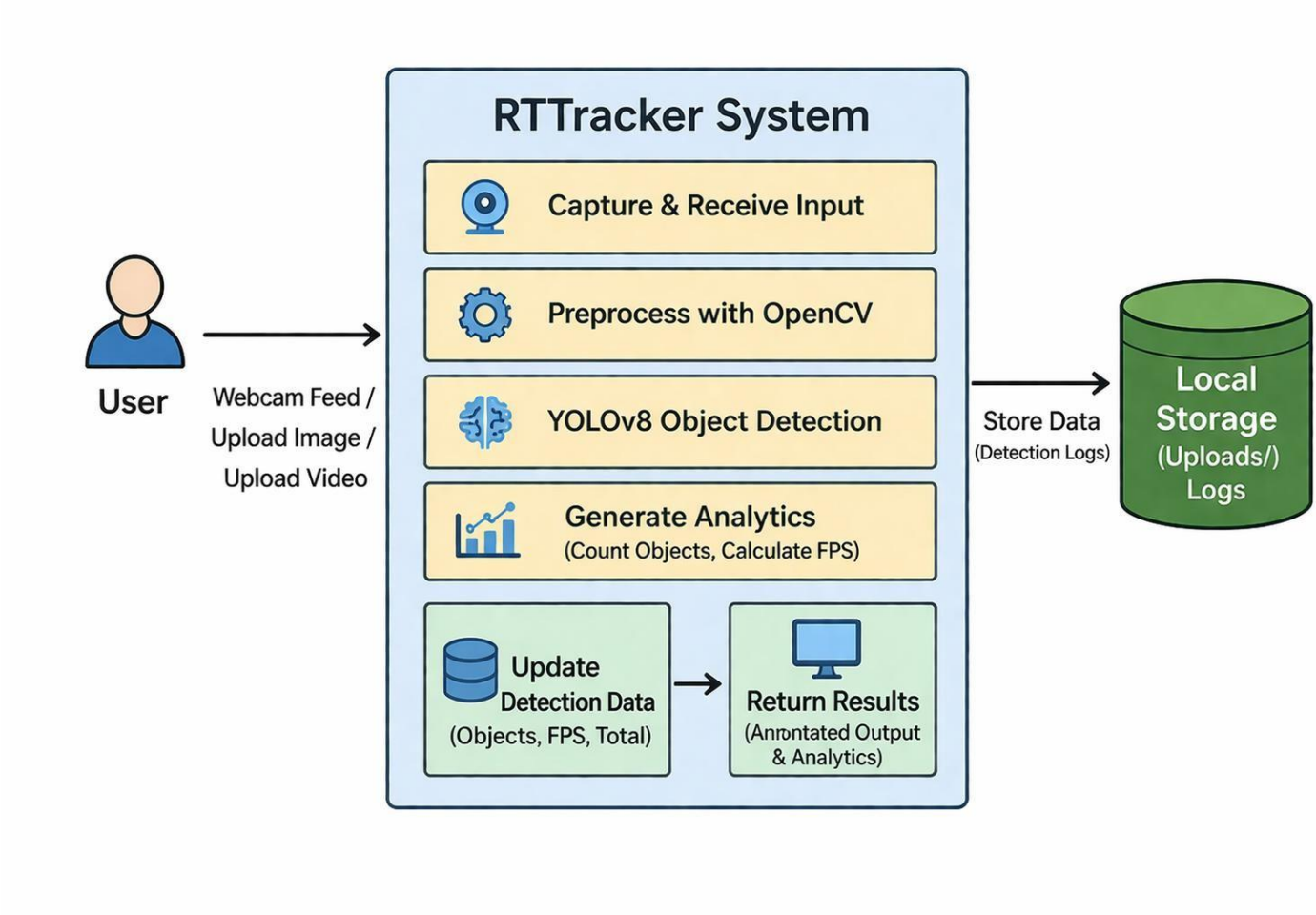

Fig: 4.2 Data Flow Diagram

4.3.1 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a use-case analysis. Its purpose is to present a graphical overview of the functionality provided by the Real-Time Object Detection System in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions such as webcam detection, image upload, video processing, and viewing results are performed for which actor. Roles of the actors in the system can be depicted.

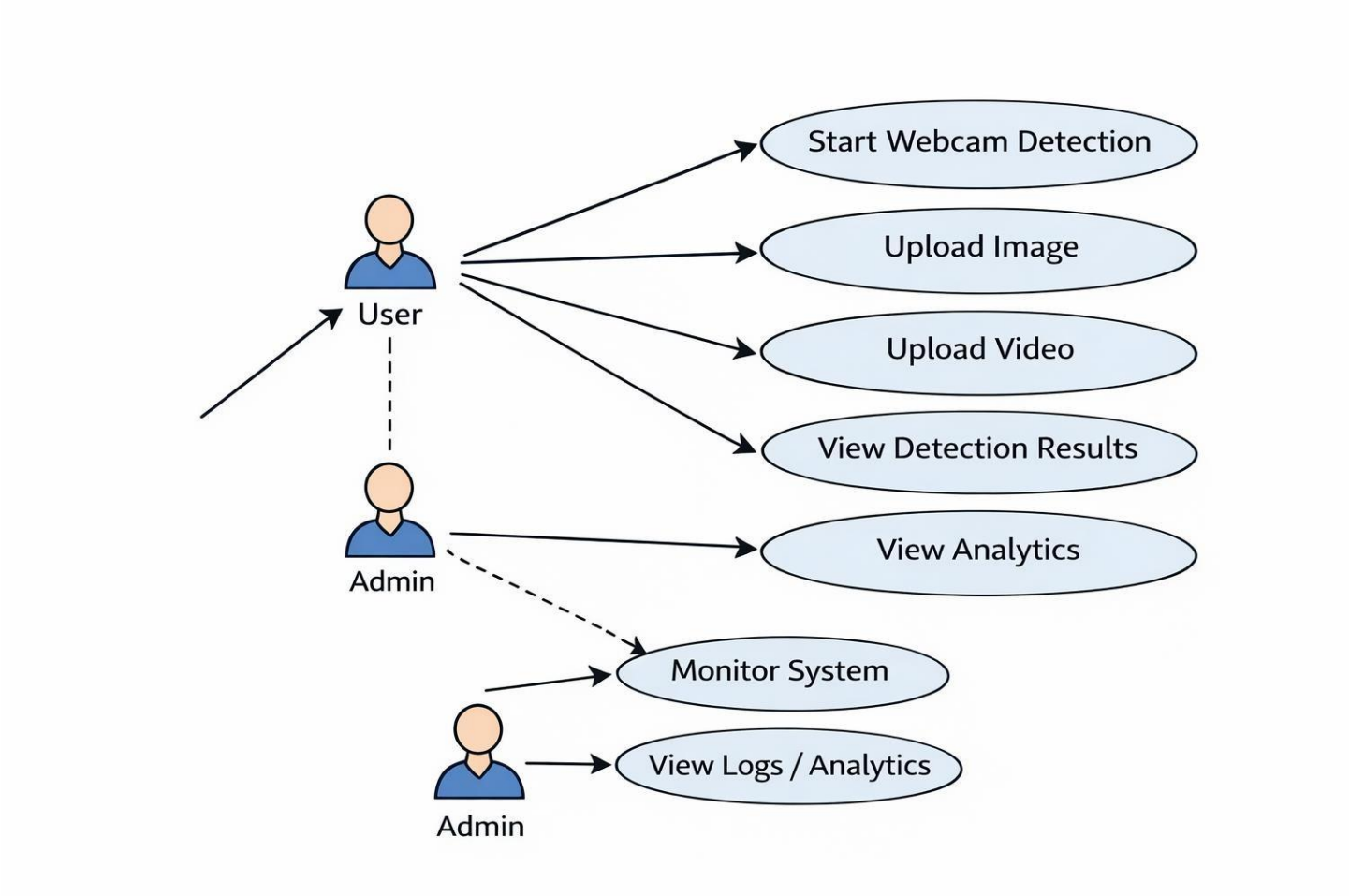

Fig 4.2 Use-Case Diagram

4.3.2 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of the Real-Time Object Detection System by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information related to object detection, user interaction, data processing, and result storage, and how these classes interact with each other to perform real-time detection and analysis.

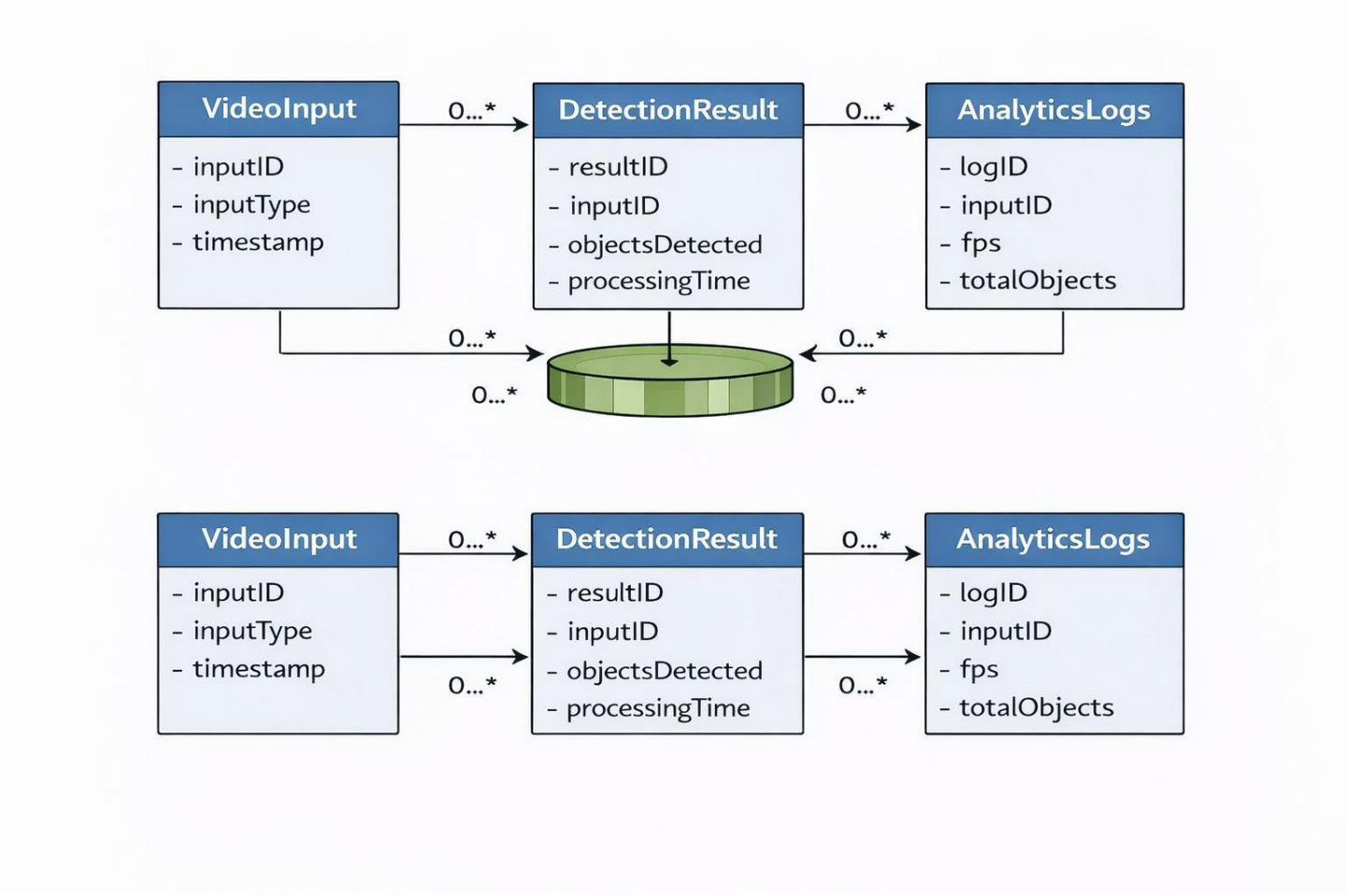

Fig 4.4 Class Diagram

4.3.3 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams

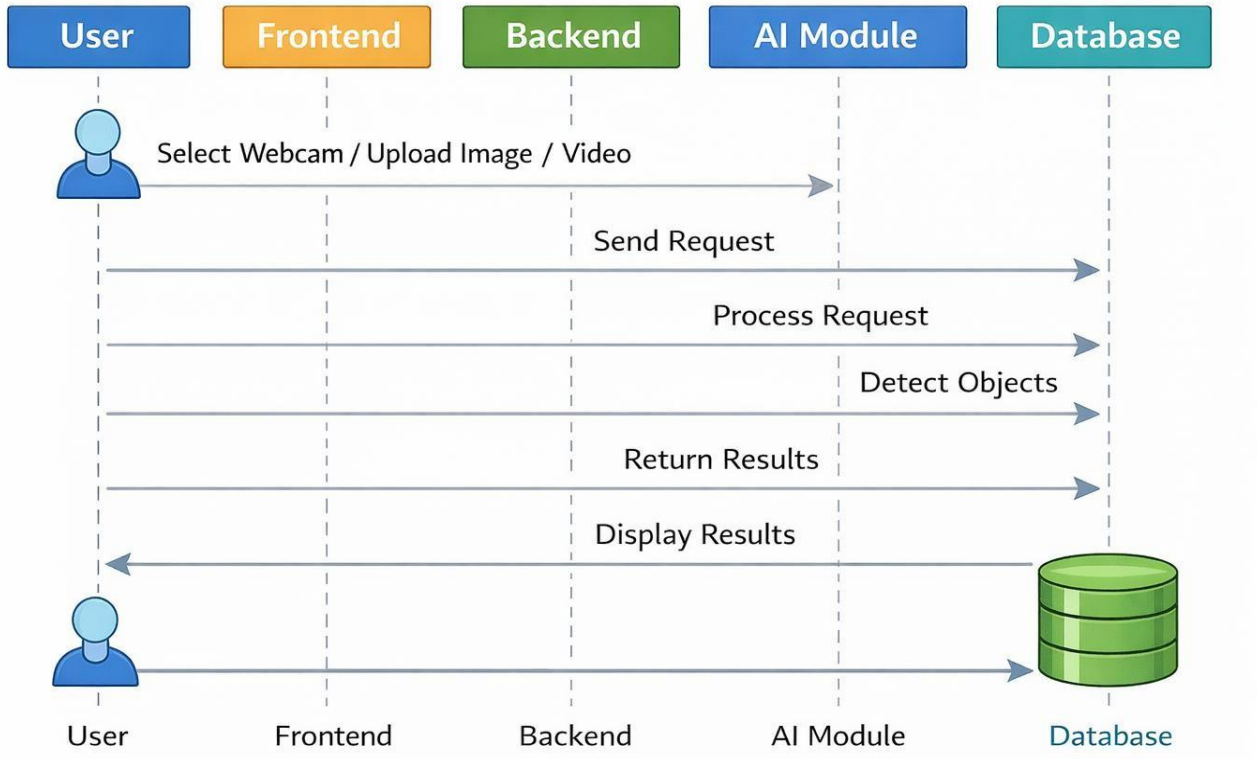

Fig 4.5 Sequence Diagram

4.3.4 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

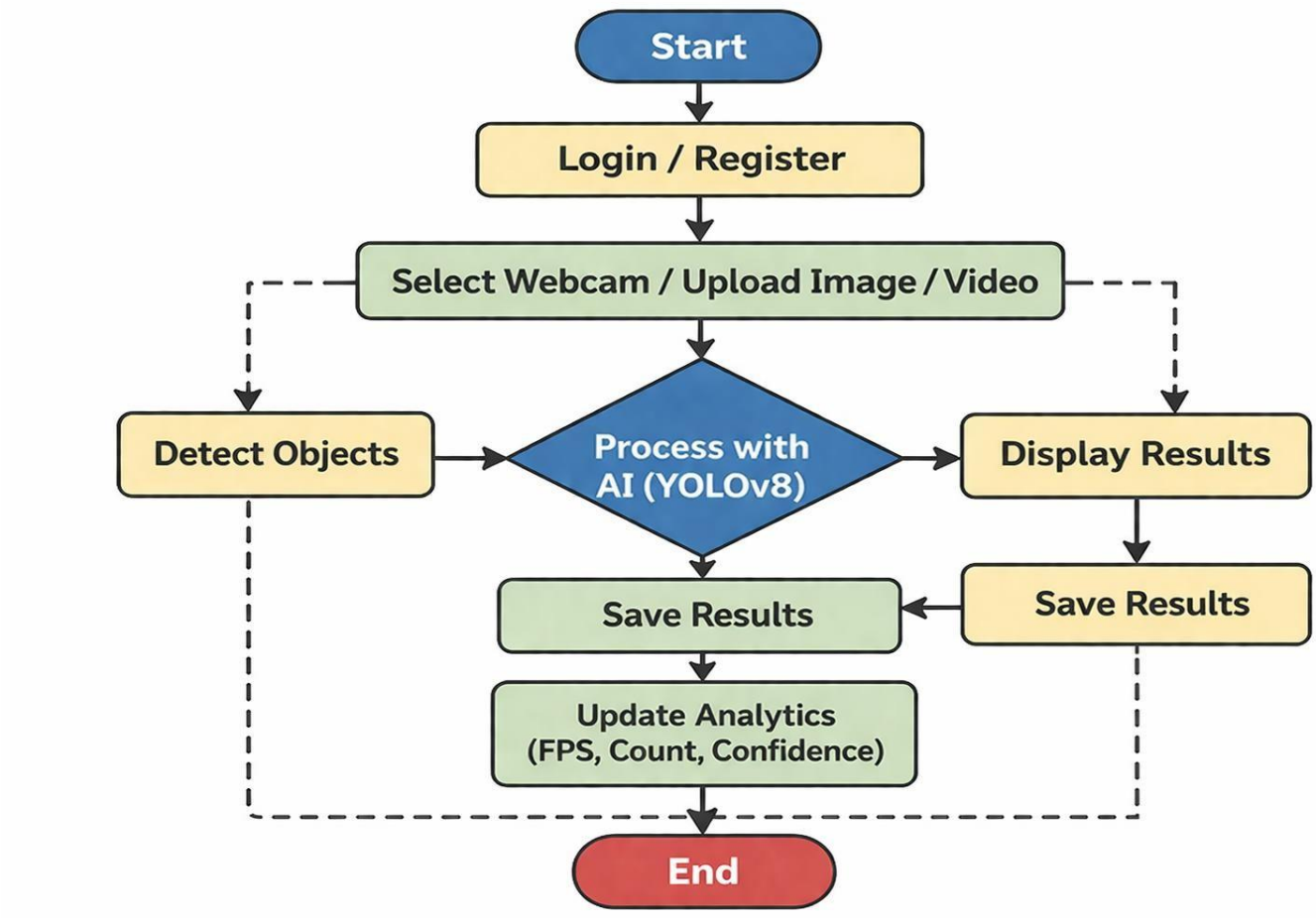

Fig: 4.6 Activity Diagram

4.4 MODULES

The system is divided into different modules such as input module, detection module, and output module. Each module performs a specific function in the system.The input module handles data input, the detection module processes data using YOLOv8, and the output module displays results and analytics on the dashboard.

.

4.4.1 Modules Description

1. User Interface Module

The User Interface Module is responsible for providing interaction between the user and the system. It allows users to select different input modes such as webcam, image upload, or video upload through a web-based dashboard. This module ensures smooth navigation and displays detection results along with analytics in a clear and user-friendly manner.

2. Webcam Detection Module

The Course Generator module is a core component of TutorX that enables users to create complete and structured courses from a simple topic input. Using advanced AI capabilities provided by Google Gemini, the module analyzes the given topic and generates detailed content, including explanations, examples, and practice questions. It organizes the information into a logical sequence of modules and subtopics, making it easier for users to follow and understand. This module eliminates the need for searching multiple resources and provides a quick and efficient way to start learning any subject.

3. Image Detection Module

The PDF Summarizer module allows users to upload study materials such as PDFs, notes, or documents and converts them into concise summaries and flashcards. It uses Natural Language Processing techniques to extract key concepts, important points, and relevant information from the uploaded content. This helps users quickly understand large volumes of text without reading everything in detail. The module also improves revision efficiency by presenting content in a simplified and structured format, making it easier for learners to retain important information.

4. Video Detection Module

The Coding Practice module provides an interactive environment where users can write, compile, and execute code in real time. It is powered by Judge0, which supports multiple programming languages and delivers instant output and error feedback. This module helps users apply theoretical knowledge to practical problems, improving their programming and problem-solving skills. By offering real-time evaluation and immediate corrections, it enhances the overall learning experience and makes coding practice more engaging and effective.

5. Object Detection Module

The Progress Tracker module monitors and records user learning activities, including completed courses, quiz performance, and coding practice results. It provides detailed insights into user progress through visual indicators and performance analytics. This module helps learners identify their strengths and weaknesses, enabling them to focus on areas that need improvement. By offering continuous feedback and tracking growth over time, it encourages consistency and helps users achieve their learning goals more efficiently.

6. analytics and storage module

It calculates and displays metrics such as object count, detection confidence, and frames per second (FPS), helping users understand system performance.It stores processed images and videos in a designated location, allowing users to access and review detection results later.

4.5 SYSTEM REQUIREMENTS

4.5.1 HARDWARE REQUIREMENTS:

Processor: Intel Core i5 or higher

RAM: Minimum: 8 GB or more

Storage: 15 GB Hard Disk

Webcam: Required for real time Detection

4.5.2 SOFTWARE REQUIREMENTS:

Operating System: Windows 10 or higher / macOS / Linux

Frontend Technologies: HTML, CSS, JavaScript

Backend Technologies: Python(Flask)

Database: MongoDB

Development Tools: Google Antigravity and Visual Studio Code, Postman (for API testing), GitHub (version control)

Libraries: OpenCV, Ultralytics, YOLOv8, NumPy, Flask

4.6 TESTING

Testing is an important phase in the development of the Real-Time Object Detection System, ensuring that the system works correctly and efficiently. It involves identifying errors, verifying functionalities, and checking whether all modules perform as expected. In this project, testing is used to validate features such as webcam detection, image processing, video analysis, and real-time analytics. It ensures that the system is reliable, user-friendly, and performs accurately before deployment.

4.6.1 Unit Testing

Unit Testing is performed on individual components of the system such as image processing, object detection, and API responses. Each module is tested separately to ensure it works correctly without errors

Use in this project:

Each module, such as the Webcam Detection, Image Detection, Video Processing, and Analytics module, is tested separately to verify correct output for given inputs.

4.6.2 Integration Testing

Integration Testing checks the interaction and data flow between different modules of the system.

Use in this project:

The communication between frontend, backend (Flask), and the YOLOv8 detection model is tested to confirm smooth data flow across the system.

4.6.3 Functional Testing

Functional Testing verifies whether the system works according to the specified requirements and performs expected functions.

Use in this project:

Features like webcam detection, image upload detection, video processing, and analytics display are tested to ensure they produce the expected results.

4.6.4 System Testing

System Testing evaluates the complete system as a whole to ensure all components work together properly.

Use in this project:

The entire Real-Time Object Detection System is tested as a whole to check overall performance, reliability, and stability.

4.6.5 White Box Testing

White Box Testing involves testing the internal logic and code structure of the system.

Use in this project:

Detection algorithms, frame processing, and backend logic are tested to ensure correct and efficient internal operations.

4.6.6 Black Box Testing

Black Box Testing tests the system based on inputs and outputs without considering internal code.

Use in this project:

User interactions, such as uploading images/videos or using webcam detection, are tested to verify that the system produces correct detection results.

4.6.7 Acceptance Testing

Acceptance Testing verifies whether the system meets user requirements and is ready for deployment.

Use in this project:

The system is evaluated in real-world scenarios to ensure all detection features work correctly and satisfy user needs.

CHAPTER 5
SOURCE CODE

App.py:

```
import os, cv2, time, base64, threading, uuid
from flask import Flask, render_template, Response, request, jsonify
from ultralytics import YOLO
from dotenv import load_dotenv
load_dotenv()
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = 'uploads'
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
# Initialize YOLO Model
def load_model(model_name):
global model
try:
print(f"Loading YOLO model: {model_name}...")
model = YOLO(model_name)
return True
except Exception as e:
print(f"Error loading model: {e}")
return False

app_config = {'model_name': 'yolov8s.pt', 'confidence': 0.45, 'iou': 0.45}
load_model(app_config['model_name'])
model_lock = threading.Lock()
@app.route('/')
def index():
    return render_template('index.html')
    return jsonify(analytics_state)
if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=True)
def process_frame(frame, tracking=True):
start_time = time.time()
if model is None: return frame, []
with model_lock:
results = model.predict(frame, conf=app_config['confidence'],
iou=app_config['iou'], verbose=False)
res = results[0]
annotated_frame = res.plot()
detected_classes = [model.names[int(box.cls[0])] for box in res.boxes] if
res.boxes else []
```

```
    # Update Analytics State
    analytics_state['fps'] = round(1.0 / (time.time() - start_time), 1)
analytics_state['objects_detected'] = {c: detected_classes.count(c) for c in
set(detected_classes)}
analytics_state['total_objects'] = len(detected_classes)
return annotated_frame, detected_classes
@app.route('/process_webcam_frame', methods=['POST'])
def process_webcam_frame():
data = request.json
img_b64 = data.get('image', '').split(',')[1]
img_data = base64.b64decode(img_b64)
import numpy as np
nparr = np.frombuffer(img_data, np.uint8)
frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
annotated_frame, detected_classes = process_frame(frame)
_, buffer = cv2.imencode('.jpg', annotated_frame)
out_b64 = base64.b64encode(buffer).decode('utf-8')
return jsonify({'image': out_b64, 'detections': list(set(detected_classes))})
```

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>RTTracker - Real-Time Object Tracking</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>  <body>
<nav class="navbar">
<span class="logo glow-text">RTTracker</span>
<ul class="nav-links">
<li><a href="#" class="nav-btn active" data-
target="simulation">Simulation</a></li>
```

```
<li><a href="#" class="nav-btn" data-target="analytics">Analytics</a></li>
<li><a href="#" class="nav-btn" data-target="history">History</a></li>
</ul>  </nav>


    <main class="content-area">
            <section id="simulation" class="page-section active">
                <h2 class="section-title">AI Detection Control Panel</h2>
                <div class="simulation-container glow-card">
                    <div class="sim-sidebar">
                        <button class="btn mode-btn active" data-
    mode="webcam">Webcam</button>
                        <button class="btn mode-btn" data-mode="image">Upload
    Image</button>
                        <div id="mode-webcam" class="mode-panel active">
                            <button class="btn btn-primary btn-glow" id="start-
    webcam">Start Camera</button>
             </div>
                    </div>
                    <div class="sim-results">
                        <div class="display-area">
                            <video id="client-webcam" autoplay
    style="display:none;"></video>
                            <img id="result-display" style="display:none;">
                            <p id="placeholder-text" class="cyber-scan">AWAITING
    FEED...</p>
                        </div>
                    </div>
                </div>
            </section>
        </main>
    </body>
    </html>
```

Main.js:

```
// Webcam Frame Capture & Processing
async function processWebcamFrame() {
if (!clientWebcam || !clientCanvas || !clientWebcam.videoWidth) return;
if (isWebcamProcessing) return;
isWebcamProcessing = true;
const ctx = clientCanvas.getContext('2d');
clientCanvas.width = 640;
const base64Data = clientCanvas.toDataURL('image/jpeg', 0.6);
{
```

```
clientCanvas.height = 480;
ctx.drawImage(clientWebcam, 0, 0, 640, 480);
try {
const response = await fetch('/process_webcam_frame', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ image: base64Data })
});
const data = await response.json();
If(data.image) {
clientWebcam.style.display = 'none';
resultDisplay.style.display = 'block';
resultDisplay.src = `${data.image}`;
}
} catch (err) {
console.error("Frame dropped:", err);
}finally{
is WebcamProcessing=False;}}
document.getElementById('start-webcam').addEventListener('click', async () =>
{
webcamStream = await navigator.mediaDevices.getUserMedia({ video: true });
clientWebcam.srcObject = webcamStream;
setInterval(processWebcamFrame, 100); // 10 FPS Processing
});
// Real-Time Analytics Heartbeat
const lineCtx = document.getElementById('lineChart');
let chart = new Chart(lineCtx, {
type: 'line',
data: {
labels: [],
datasets: [{
label: 'Total Targets',
data: [],
borderColor: '#00ffff',
fill: true,
tension: 0.4
}]
},
options: { animation: { duration: 0 } }
});
function updateDashboard(data) {
const total = data.total_objects || 0;
const timeLabel = new Date().toLocaleTimeString();
```

```
// Rolling Chart Logic
chart.data.labels.push(timeLabel);
chart.data.datasets[0].data.push(total);
if (chart.data.labels.length > 20) {
chart.data.labels.shift();
chart.data.datasets[0].data.shift();
}
chart.update('none');
// Stats Counter Updates
document.getElementById('val-objects').textContent = total;
document.getElementById('val-fps').textContent = data.fps.toFixed(1);
}
setInterval(() => {
fetch('/analytics_data').then(r => r.json()).then(updateDashboard);
}, 2000);
// Dynamic Monitor Grid Logic
const monitorGrid = document.getElementById('dynamic-monitor-grid');
let monitorCount = 0;
function addMonitorFeed(sourceType, sourceData) {
monitorCount++;
const monitorId = `monitor-${monitorCount}`;
const monitorHTML = `
<div class="monitor-card glow-card" id="${monitorId}">
<div class="monitor-header">
<span class="status-dot pulse"></span>
<span class="monitor-title">FEED ${monitorCount}</span>
</div>
<div class="monitor-screen">
<img src="${sourceData}" class="feed-active">
</div>
<div class="monitor-stats">
<div class="obj-chips" id="chips-${monitorId}"></div>
</div>
</div>`;
monitorGrid.insertAdjacentHTML('beforeend', monitorHTML);
const alertTargets = ['person', 'knife', 'backpack'];
function checkAlerts(detectedClasses) {
const activeAlerts = detectedClasses.filter(cls =>
alertTargets.includes(cls));
if (activeAlerts.length > 0) {
const log = document.getElementById('surveillance-logs');
const alertMsg = `<div class="log-entry threat">
```

```
<span class="time">[${new Date().toLocaleTimeString()}]</span>
<span class="msg">CRITICAL: ${activeAlerts.join(', ')} detected!</span>
</div>`;
log.insertAdjacentHTML('afterbegin', alertMsg);
}}
document.body.classList.add('alert-mode');
setTimeout(() => document.body.classList.remove('alert-mode'), 1000);
```

Cloud Integration & Database Logic (app.py)

```
from supabase import create_client, Client
# Initialize Supabase Client
supabase_url = os.environ.get("SUPABASE_URL")
supabase_key = os.environ.get("SUPABASE_KEY")
supabase: Client = create_client(supabase_url, supabase_key) if supabase_url
and supabase_key else None
@app.route('/log_event', methods=['POST'])
def log_event():
user_id = get_auth_user() # Verify Clerk JWT Token
if not supabase: return jsonify({'error': 'Supabase not configured'}), 500
data = request.json
obj_list = data.get('objects', [])
# Store critical metadata in the cloud
insert_data = {
"user_id": user_id,
"media_type": data.get('source', 'Webcam'),
"total_objects": len(obj_list),
"fps": 0.0,
"objects_detected": {c: obj_list.count(c) for c in set(obj_list)},
"created_at": "now()"
}
supabase.table("detection_events").insert(insert_data).execute()
    return jsonify({'success': True})
@app.route('/history')
def get_history():
    user_id = get_auth_user()
response = supabase.table("detection_events") \
        .select("*") \
        .eq("user_id", user_id) \
        .order("created_at", desc=True) \
        .execute()
    return jsonify(response.data)
```

CHAPTER 6
EXPERIMENTAL RESULTS

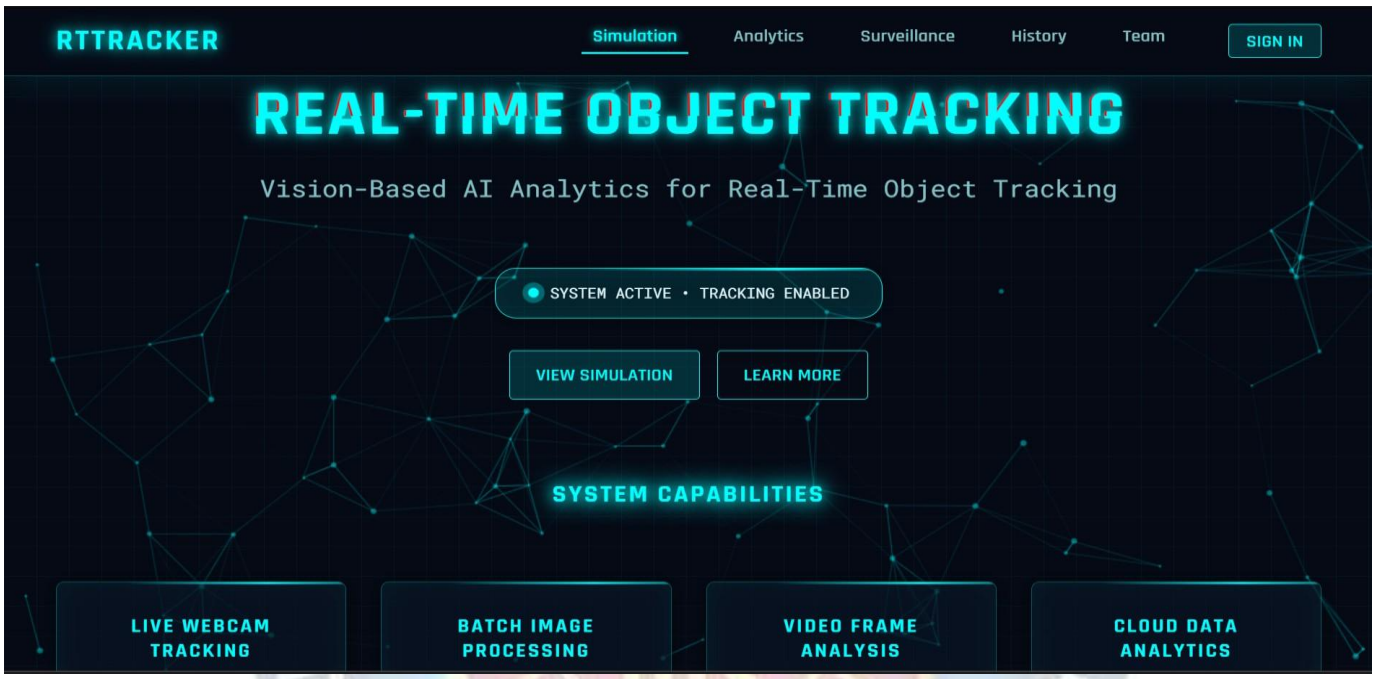

Fig 6.1 Home Dashboard Page

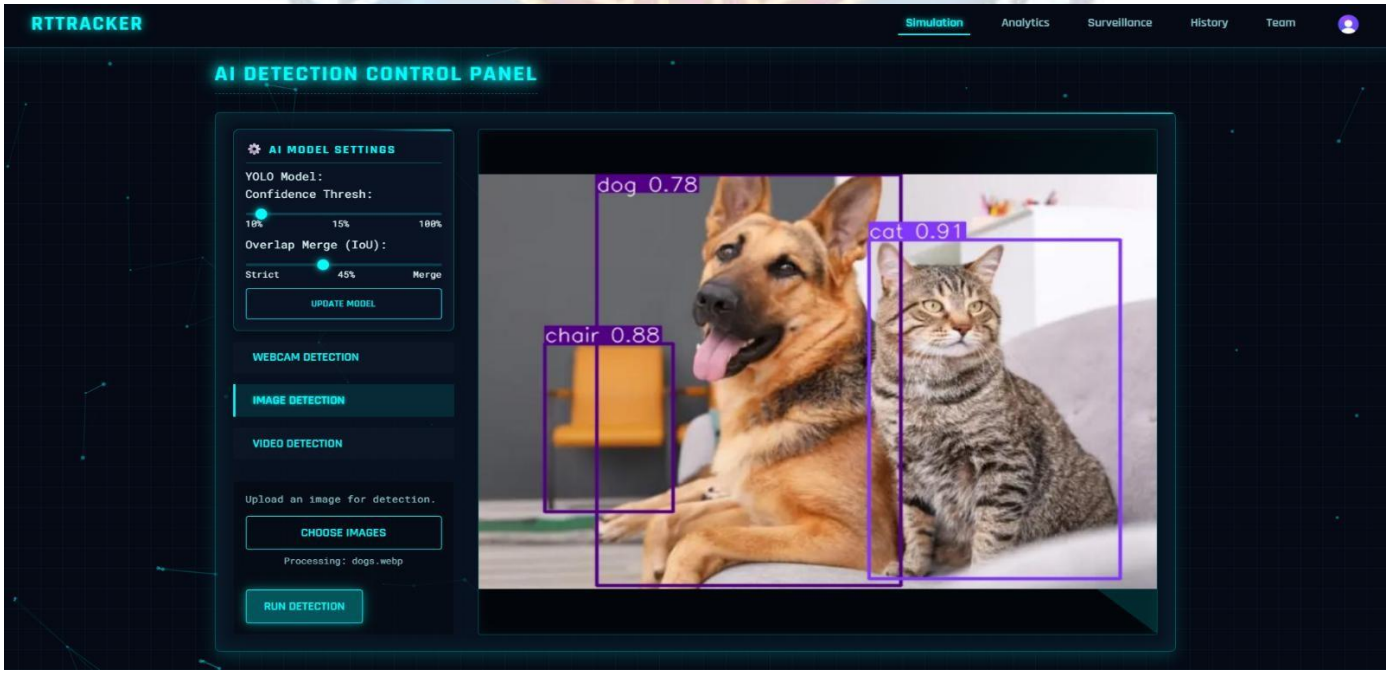

Fig 6.2 Image Detection Page

Fig 6.3 Video Detection Interface

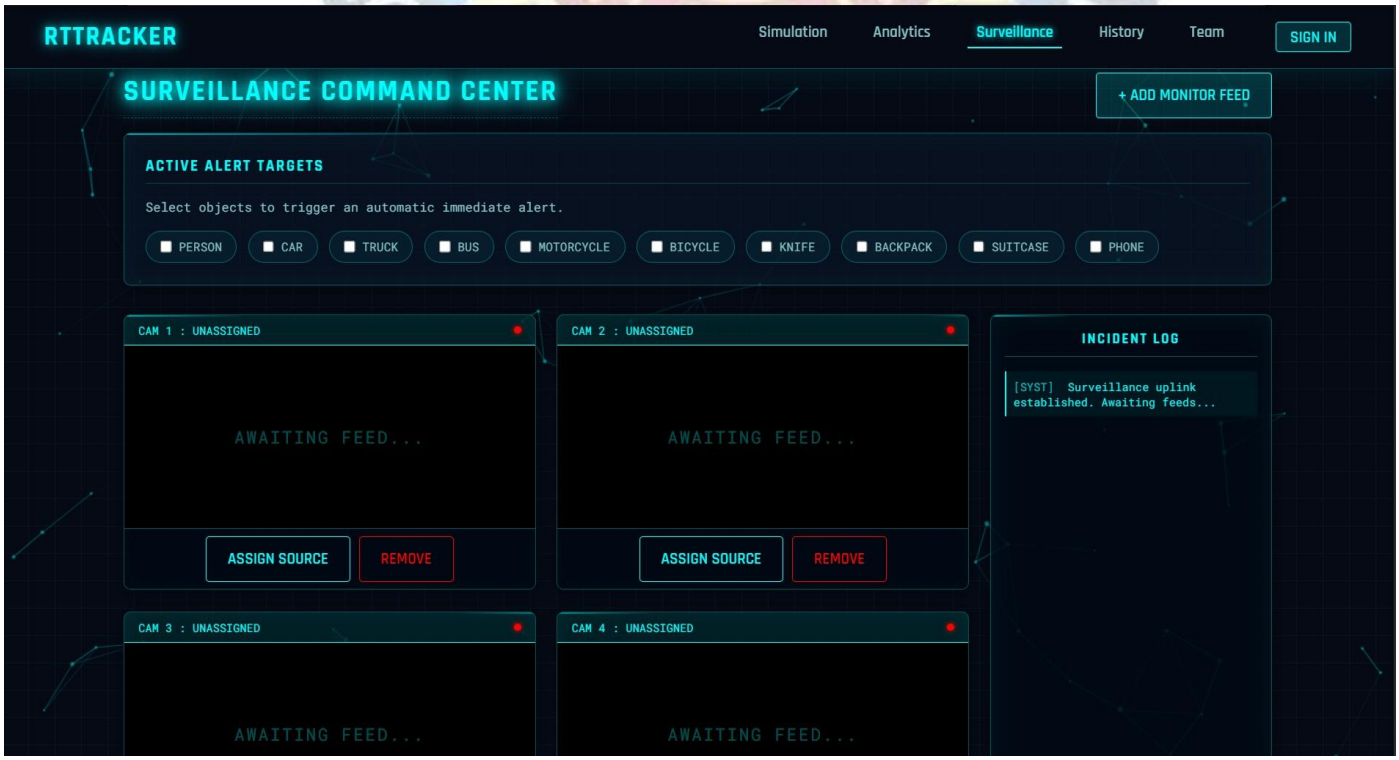

Fig 6.4 Survillance Page

Fig 6.5 Analytics Page

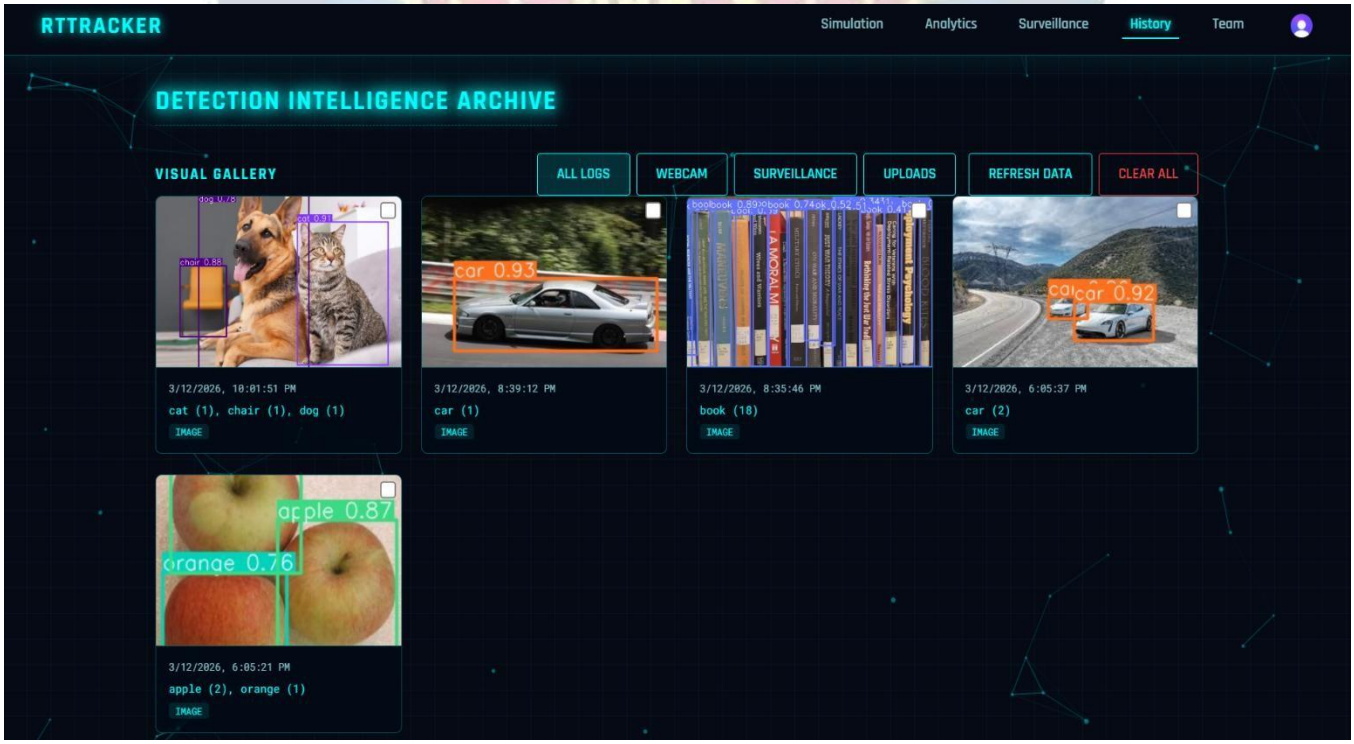

Fig 6.6 History Page

CHAPTER 7
CONCLUSION &FUTURE ENHANCEMENT

7.1 CONCLUSION

The Real-Time Object Detection System represents a significant advancement in the field of computer vision by integrating artificial intelligence, real-time processing, and user-friendly web technologies into a single unified platform. Unlike traditional monitoring systems that rely on manual observation or basic video recording, this system provides automated and intelligent object detection capabilities. By utilizing the YOLOv8 deep learning algorithm, the system can accurately detect and classify multiple objects in real time, enabling users to analyze visual data efficiently without constant human supervision. This approach not only reduces manual effort but also improves accuracy and speed in detection tasks.

The system also emphasizes practical usability by supporting multiple input sources such as webcam, image upload, and video processing. These features allow users to perform detection in different scenarios, making the system flexible and adaptable. The integration of real-time analytics such as object count, confidence scores, and frames per second (FPS) enhances the user experience by providing meaningful insights into system performance. Additionally, the system visually highlights detected objects with bounding boxes and labels, making the results easy to understand and interpret.

The platform ensures efficiency and scalability through a well-structured backend using Flask and optimized computer vision libraries such as OpenCV and YOLO. It is capable of processing continuous data streams and handling multiple detection tasks effectively. Overall, the Real-Time Object Detection System demonstrates how artificial intelligence can be applied to automate monitoring, improve accuracy, and provide intelligent insights, making it a powerful solution for applications such as surveillance, traffic monitoring, and smart systems.

.

7.2 FUTURE ENHANCEMENT

Although the Real-Time Object Detection System already provides an efficient and intelligent solution for detecting objects using artificial intelligence, there are several opportunities to enhance its functionality and performance in the future. One significant improvement is the integration of advanced tracking algorithms that can not only detect objects but also track their movement across frames. This would enable applications such as behavior analysis, motion tracking, and automated surveillance. Additionally, incorporating voice-based alerts or notifications can help users receive real-time warnings when specific objects are detected, improving usability in safety-critical environments.

Another important enhancement is the development of a mobile application for the system. A mobile app would allow users to access object detection features anytime and anywhere, increasing flexibility and user engagement. Real-time notifications could be implemented to alert users when certain objects are detected or when abnormal activity occurs. Furthermore, integrating cloud storage and database systems would allow users to store detection history, logs, and analytics data for future reference and detailed analysis.

Expanding the system to support multi-camera integration and IoT-based devices is another valuable enhancement, enabling large-scale monitoring in smart cities, industries, and security systems. Future improvements could also include training custom object detection models for specific use cases, improving accuracy for domain-specific applications. Additionally, incorporating advanced analytics, heatmaps, and predictive insights can further enhance system intelligence. By implementing these improvements, the Real-Time Object Detection System can evolve into a more powerful, scalable, and intelligent solution for real-world applications.

REFERENCES

[1] R. Sharma, A. Kumar, and S. Verma, "Real-Time Object Detection using Deep Learning Techniques," Proceedings of the Indian Conference on Computer Vision, pp. 120–128, 2023.

International Journal of Emerging Technologies in Learning (iJET), pp. 40–48, 2024.

[2] P. Reddy and K. Singh, "YOLO-Based Object Detection for Smart Surveillance Systems," International Journal of Computer Science and Engineering, pp. 45–52, 2024.

[3] Ultralytics, "YOLOv8 Documentation," 2024. [Online]. Available: https://docs.ultralytics.com

[4] S . Gupta and M. Agarwal, "Computer Vision Techniques using OpenCV," Indian Journal of Technology and Engineering, pp. 210–218, 2022.

[5] N. OpenCV, "Open Source Computer Vision Library Documentation," 2024. [Online]. Available: https://opencv.org

[6] Python Software Foundation, "Python Documentation," 2024. [Online]. Available: https://docs.python.org

[7] Flask, "Flask Web Framework Documentation," 2024. [Online]. Available: https://flask.palletsprojects.com

[8] N. Patel and R. Joshi, "Efficient Image Processing using NumPy," International Journal of Advanced Computing, pp. 78–85, 2023.

[9] "Roboflow, "Object Detection Dataset and Model Training Platform," 2024. [Online]. Available: https://roboflow.com

[10] GitHub, "GitHub Documentation," 2024. [Online]. Available: https://docs.github.com

[11] Express.js, "Express.js Documentation," 2024. [Online]. Available: https://expressjs.com

[12] Mozilla Developer Network, "MDN Web Docs," 2024. [Online]. Available: https://developer.mozilla.org

[13] Visual Studio Code, "VS Code Documentation," 2024. [Online]. Available: https://code.visualstudio.com/docs

[14] "Antigravity Concepts and Future AI-Based Systems," 2025. [Online]. Available: https://ieeexplore.ieee.org